RFC1350 Learning Report

Revision History

Revision	Author	Contact Info	Description	Date
1.0	Yamin		First revision of RFC1350 Learning Report	25-10-23

1. INTRODUCTION

1.1 Purpose:

TFTP, the Trivial File Transfer Protocol, serves the primary purpose of transferring files between networked machines. It operates over the Internet User Datagram Protocol (UDP), making it suitable for moving files between machines on different networks that support UDP. TFTP is designed to be lightweight and easy to implement, emphasizing simplicity over advanced features.

Unlike regular FTP, TFTP has limited capabilities and can only perform file (or mail) read and write operations with a remote server. Notably, it does not support directory listing or user authentication. TFTP passes data in 8-bit bytes and supports three transfer modes: netascii (modified ASCII), octet (raw 8-bit bytes), and mail (sending netascii characters to a user). The mail mode is considered obsolete, and TFTP can potentially support additional modes defined by cooperating hosts.

1.2 Layer in TCP/IP or OSI Model

TFTP operates at the Application Layer of the OSI model or within the Transport Layer of the TCP/IP model. It uses the User Datagram Protocol (UDP) for transport, which is a connectionless protocol.

The TFTP server continuously listens for requests on the well-known UDP port number 69, which is reserved for TFTP. The client selects an ephemeral port number for its initial communication, as is typical in TCP/IP. This port number serves as the transfer identifier (TID), identifying the data transfer.

The server, however, uses a pseudo-random TID for sending responses back to the client, rather than using port number 69. This approach allows multiple TFTP exchanges to occur simultaneously by a single server.

1.3 Main Procedure

TFTP operation consists of three general steps: initial connection, data transfer, and connection termination. All operations are performed through the exchange of specific TFTP messages.

1. **Request to Read or Write**: Any transfer begins with a request to read (RRQ) or write (WRQ) a file. These requests also serve to request a connection with the server. The client specifies the filename to be read or written and the desired transfer mode (netascii or octet).

2. **Positive Reply and Acknowledgment:** If the server grants the request, it responds with a positive reply, which implicitly establishes the logical TFTP connection.

In the case of a read request (RRQ), the server immediately sends the first data message.In the case of a write request (WRQ), the server sends an acknowledgment message, allowing the client to proceed to send the first data message.

3. **Data Transfer:** Data and acknowledgment messages are exchanged in a lock-step manner.
For a read (RRQ), the server sends one data message and waits for an acknowledgment from the client before sending the next one.

For a write (WRQ), the client sends one data message, and the server sends an acknowledgment for it before the client sends the next data message. Data messages carry blocks of data, with each block being sequentially numbered (starting from 1) and can contain up to 512 bytes of data.

4. **Connection Termination:** The receipt of a data message with between 0 and 511 bytes of data signals the last data message. Once this message is acknowledged, it automatically signals the end of the data transfer. There is no need to explicitly terminate the "connection."

1.4 Packet Formats

TFTP itself has five different message types that it can use. They are as follows:

1. **RRQ**, which breaks out as Read Request.
2. **WRQ**, which breaks out as Write Request.
3. **DATA**, which breaks out as, to read or write the next chunk of data.
4. **ERROR**, which breaks out as Error message.
5. **ACK**, which breaks out as Acknowledgement.

Table -1: TFTP Read Request / Write Request Packet Format

<table>
<tr><th>Field Name</th><th>Size (bytes)</th><th colspan="2">Description</th></tr>
<tr><td>Opcode</td><td>2</td><td colspan="2">Operation Code: Specifies the TFTP message type. A value of 1 indicates a Read Request message, while a value of 2 is a Write Request message.</td></tr>
<tr><td>Filename</td><td>Variable</td><td colspan="2">Filename: The name of the file to be read or written.</td></tr>
<tr><td>Mode</td><td>Variable</td><td colspan="2">Transfer Mode: The string “netascii” or “octet”, zero-terminated.</td></tr>
<tr><td rowspan="3">Option</td><td rowspan="3">Variable</td><td>Subfiled Name</td><td>Description</td></tr>
<tr><td>OptN</td><td>A string specifying the name of the option. Such as “blksize”, “interval”, “tsize”.</td></tr>
<tr><td>valueN</td><td>The value the client is requesting for this option.</td></tr>
</table>

```
+-------+---~~---+---+---~~---+---+---~~---+---+---~~---+---+-->
| opc   |filename| 0 | mode   | 0 | opt1   | 0 | value1 | 0 | <
+-------+---~~---+---+---~~---+---+---~~---+---+---~~---+---+-->
>-------+---+---~~---+---+
< optN  | 0 | valueN | 0 |
>-------+---+---~~---+---+
```

Figure 1-1: RRQ/WRQ packet

Table-2: TFTP Data Packet format

Field Name	Size (bytes)	Description
Opcode	2	Operation Code: Specifies the TFTP message type. A value of 3 indicates a Data packet.
Block #	2	Block Number: The number of the data block being sent.
Data	Variable	Data: 0 to 512 bytes of data.

```
  2 bytes    2 bytes    n bytes
-----------------------------------
 | Opcode  |  Block #   | Data |
-----------------------------------
```

Figure 1-2: DATA packet

Table 3: TFTP Acknowledgment packet Format

Field Name	Size (bytes)	Description
Opcode	2	Operation Code: Specifies the TFTP message type. A value of 4 indicates an Acknowledgment message.
Block #	2	Block Number: The number of the data block being acknowledged; a value of 0 is used to acknowledge receipt of a write request without options, or to acknowledge receipt of an option acknowledgment.

```
  2 bytes    2 bytes
-------------------------
| Opcode  |   Block # |
-------------------------
```

Figure 1-3: ACK packet

Table 4. TFTP Error packet Format

Field Name	Size (bytes)	Description
Opcode	2	Operation Code: Specifies the TFTP message type. A value of 5 indicates an Error message.
Error Code	2	Value — Meaning 0 — Not defined, see error message (if any). 1 — File not found. 2 — Access violation. 3 — Disk full or allocation exceeded. 4 — Illegal TFTP operation. 5 — Unknown transfer ID. 6 — File already exists. 7 — No such user.
	Variable	Error Message: A descriptive text error message string, intended for "human consumption", as the standard puts it.

```
  2 bytes   2 bytes    string   1 byte
 ----------------------------------------
 | Opcode |  ErrorCode | ErrMsg   | 0 |
 ----------------------------------------
```

Figure 1-4: ERROR packet

Table 5: TFTP Option Acknowledgment Packet Format

<table>
<tr><th>Field Name</th><th>Size (bytes)</th><th colspan="2">Description</th></tr>
<tr><td>Opcode</td><td>2</td><td colspan="2">Operation Code: Specifies the TFTP message type. A value of 6 indicates an Option Acknowledgment message.</td></tr>
<tr><td rowspan="3">Option</td><td rowspan="3">Variable</td><td>Subfiled Name</td><td>Description</td></tr>
<tr><td>OptN</td><td>A string specifying the name of the option. Such as "blksize", "interval", "tsize".</td></tr>
<tr><td>valueN</td><td>The value the client is requesting for this option.</td></tr>
</table>

```
+-------+---~~---+---+---~~---+---+---~~---+---+---~~---+---+
 | opc  | opt1   | 0 | value1 | 0 | optN   | 0 | valueN | 0 |
+-------+---~~---+---+---~~---+---+---~~---+---+---~~---+---+
```

Figure 1-5: OACK Packet

1.5 Main Mechanism

1. **Read and Write Requests**: TFTP begins with a client sending a read request (RRQ) or a write request (WRQ) to a server. The request includes the filename and transfer mode (e.g., netascii, octet).

2. **Data Transfer**: TFTP divides the file into fixed-length data blocks of 512 bytes (or less in the final block). The server responds to a read request with data packets, and the client acknowledges each data packet with an ACK packet. For write requests, the client sends data packets, and the server acknowledges them. This data transfer process is lock-step, with one data packet being acknowledged before the next is sent.

3. **Connection Termination**: The connection is terminated when a data packet contains less than 512 bytes of data, signaling the end of the file. The client acknowledges this last packet, and the server knows the transfer is complete. If a timeout occurs, indicating the loss of a packet, re-transmissions are performed.

4. **Error Handling**: If there are issues with the request, transfer, or access to resources, TFTP responds with error packets, which may lead to the termination of the connection. There's an acknowledgment for error packets, which can be retransmitted. An example of an error is a file not found, access violation, or disk full.

5. **Random Port Assignment**: Each end of the connection selects a random port (TID - Transfer Identifier) to be used for the duration of the connection. This avoids conflicts and helps in distinguishing different connections.

6. **Option Acknowledgment**: RFC 2347 introduces the OACK packet, which allows clients and servers to negotiate options, such as block size. This mechanism enhances the protocol's flexibility and efficiency.

7. **Window Size Negotiation**: RFC 2349 introduces the concept of window size negotiation to allow for larger data block windows, improving overall transfer efficiency and performance.

2. KEY POINTS OF CONTENT

2.1 Interaction Sequence Diagrams

Client initiates a read request to the server, who responds with data packets containing blocks of the file. The client acknowledges each block, and when a short block is acknowledged, it marks the file as complete. The server acknowledges the final acknowledgment, indicating a successful file transfer.

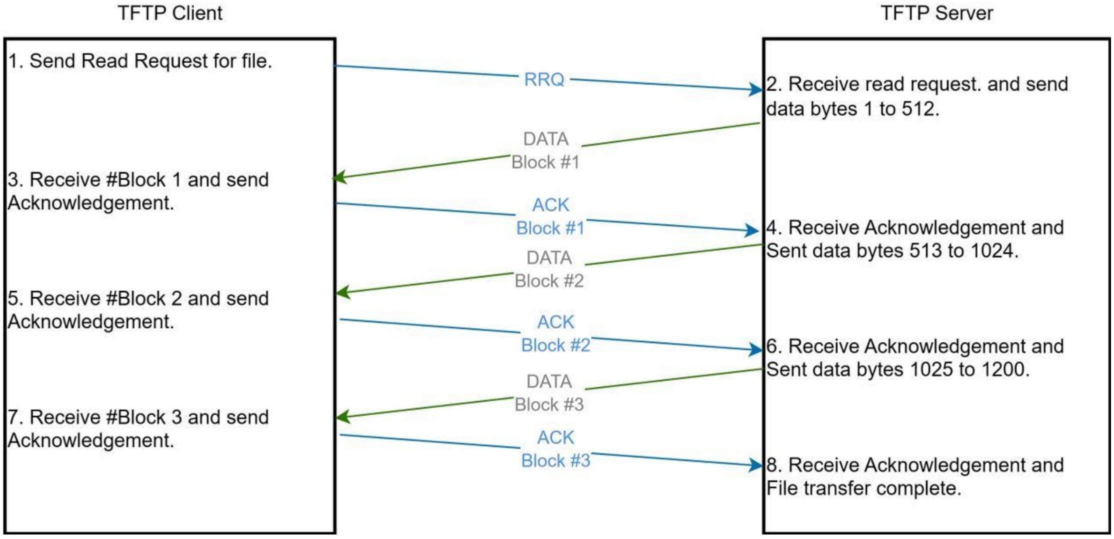

Figure 2-1: TFTP Read Process

Client sends a write request, server acknowledges it with block #0. Client sends data in blocks, server acknowledges each. The transfer ends when a short block is acknowledged, indicating the successful completion of the write process.

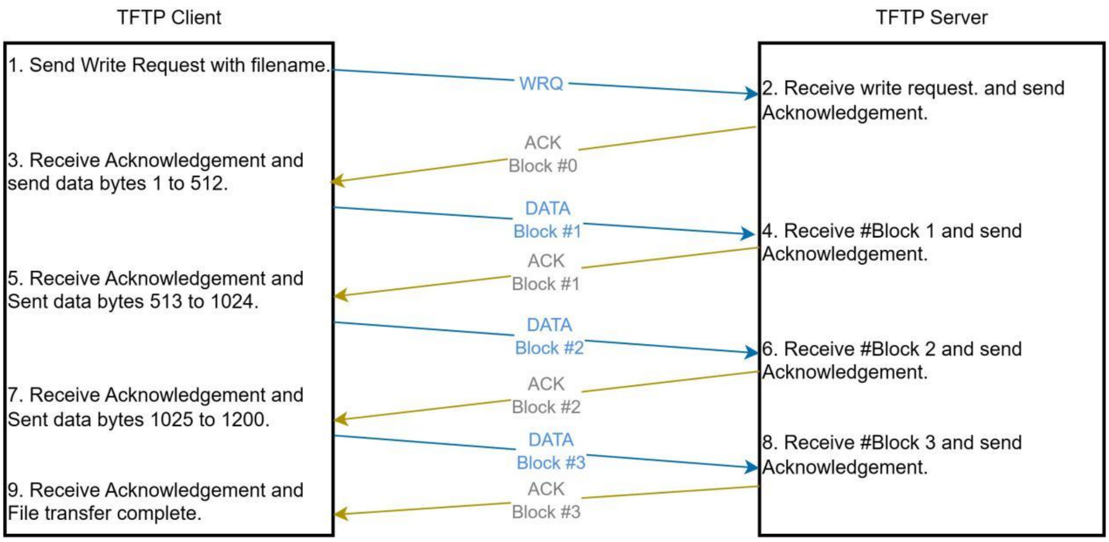

Figure 2-2: TFTP Write Process

For a normal read without option negotiation, the client sends a Read Request, and the server responds with data blocks and acknowledgments. With option negotiation, the client sends a Read Request with options, receives an Option Acknowledgment from the server, and proceeds with data blocks and acknowledgments.

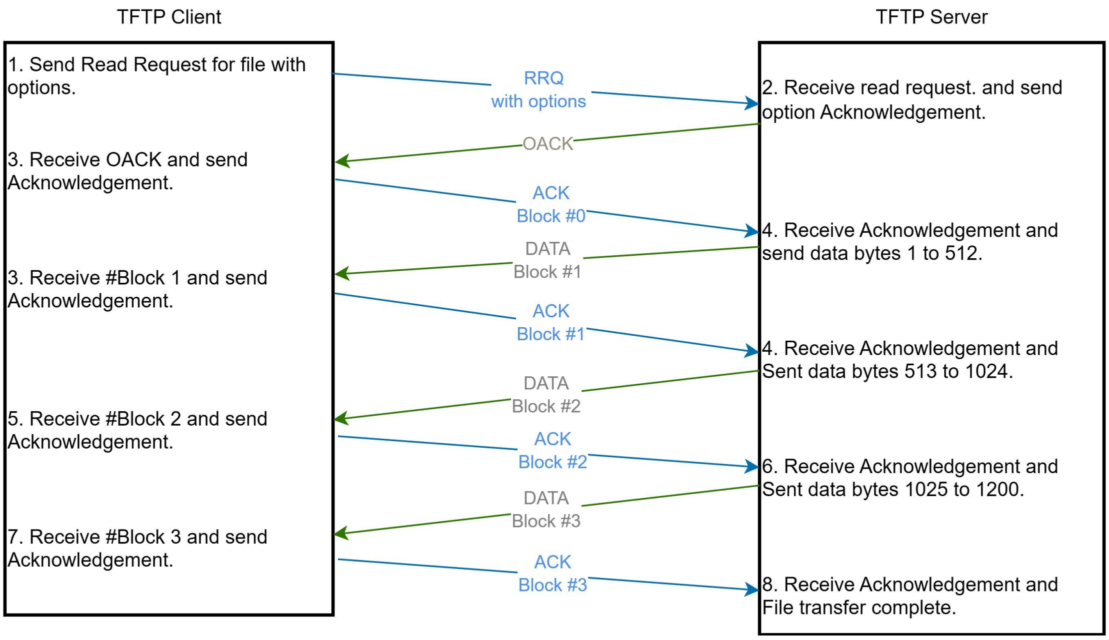

Figure 2-3: TFTP Read Process With Option Negotiation

DSND (Data Sender) sends a series of consecutive data blocks (DB) to DRCV according to the agreed window size. DRCV (Data Receiver) acknowledges the last received data block with an ACK. If an ACK is delayed or times out, DSND begins the next window with the last received ACK. In the event of a data block sequence error, DRCV notifies DSND by sending an ACK for the last correctly received data block. DSND responds by sending a new window of data blocks, starting based on the out-of-sequence ACK received from DRCV.

This diagram shows the flow of data and acknowledgments between the sender and receiver, demonstrating how window size and error handling are managed in the TFTP transfer.

```
[ DRCV ]   <---traffic--->       [ DSND ]
 ACK#              ->          <- Data Block#       window block#
                    ...
                   <-             |DB n+01|             1
                   <-             |DB n+02|             2
                   <-             |DB n+03|             3
                   <-             |DB n+04|             4
|ACK n+04|         ->
                   <-             |DB n+05|             1
             Error |<-            |DB n+06|             2
                   <-             |DB n+07|             3
|ACK n+05|         ->
                   <-             |DB n+06|             1
                   <-             |DB n+07|             2
                   <-             |DB n+08|             3
                   <-             |DB n+09|             4
|ACK n+09|         ->
                   <-             |DB n+10|             1
             Error |<-            |DB n+11|             2
                   <-             |DB n+12|             3
|ACK n+10|         ->| Error
                   <-             |DB n+13|             4
                                - timeout -
                   <-             |DB n+10|             1
                   <-             |DB n+11|             2
                   <-             |DB n+12|             3
                   <-             |DB n+13|             4
|ACK n+13|         ->
                    ...
```

Figure 2-4: window size negotiation and error handling

2.2 Configurable Parameters

1. **Transfer Mode**:

The transfer mode refers to the format used for file transfers. TFTP supports two modes:

- NetASCII Mode: This mode is used for transferring text files, and it requires the server to perform character set translation (e.g., from ASCII to EBCDIC).
- Octet Mode: This mode is used for binary files and transfers data as-is, without character set translation.
- Mail Mode: Used for sending files to email recipients, specifying recipient names.

2. **Timeout and Re-transmission:**

TFTP uses timeout and re-transmission mechanisms to ensure reliable data transfer. The following parameters are related to this:

- **Timeout Interval**: The time a sender waits for an acknowledgment (ACK) before re-transmitting the data packet.
- **Maximum Re transmission Attempts**: The maximum number of times a sender will re-transmit a data packet if it doesn't receive an acknowledgment.

3. **Block Size:**

The block size defines the number of data bytes in each data packet. The standard block size in TFTP is 512 bytes. RFC 2349 allows for variable block sizes, enabling larger packets for faster transfers over reliable networks.

4. **Transfer Size:**

The transfer size option in TFTP is specified as an extension to the Read Request (RRQ) and Write Request (WRQ) packets. The “tsize” option can be used to specify the size of the file to be transferred. If the file size is too large for the server to handle, it may abort the transfer with an Error packet.

5. **Window Size:**

The window size is a configurable parameter that specifies the number of data packets a sender can transmit before waiting for acknowledgments from the receiver. It allows for more efficient data transfer by reducing acknowledgment overhead. The default window size is typically 1.

2.3 Key Point C

3. QUESTIONS

3.1 Question 1

3.2 Question 2